

GUÍA DE REFERENCIA DE CÓDIGO — FinCredit: Credit & Loan Management System

Proyecto — Autores: Henry Garrido - Cristian Castro | Fecha: 29-05-2026

ESTRUCTURA DE CARPETAS

```
src/
├── com/fincredit/
│   ├── gui/           → Interfaz gráfica (Swing)
│   ├── interfaces/    → Interfaces IEvaluable, ILoanable
│   ├── logic/         → Lógica de negocio (evaluador, fábrica)
│   ├── model/         → Clases del dominio (Person, Client, Loan...)
│   └── registry/      → Repositorio central de datos (LoanRegistry)
└── module-info.java
```

PILARES POO — DÓNDE ENCONTRARLOS

1. HERENCIA (extends)

Qué hereda	Clase hija	Clase padre	Archivo	Línea
Client extends Person	Client	Person (abstracta)	model/Client.java	L16
LoanOfficer extends Person	LoanOfficer	Person (abstracta)	model/LoanOfficer.java	L13
ConsumerLoan extends LoanProduct	ConsumerLoan	LoanProduct (abstracta)	model/ConsumerLoan.java	L9
EducationalLoan extends LoanProduct	EducationalLoan	LoanProduct (abstracta)	model/EducationalLoan.java	L9
MortgageLoan extends LoanProduct	MortgageLoan	LoanProduct (abstracta)	model/MortgageLoan.java	L14

FreeInvestmentLoan extends LoanProduct	FreeInvestmentLoan	LoanProduct (abstracta)	model/FreeInvestmentLoan.java	L9
LoanProduct implements ILoanable	LoanProduct	ILoanable	model/LoanProduct.java	L15
CreditEvaluator implements IEvaluable	CreditEvaluator	IEvaluable	logic/CreditEvaluator.java	L13

2. POLIMORFISMO

Método polimórfico	Dónde se define	Dónde se sobreescribe (@Override)	Archivo	Línea
getRole()	Person abstracto	Client → "CLIENT"	model/Client.java	L33
getRole()	Person abstracto	LoanOfficer → "LOAN_OFFICER"	model/LoanOfficer.java	L28
getSummary ()	Person abstracto	Client	model/Client.java	L38–44
getSummary ()	Person abstracto	LoanOfficer	model/LoanOfficer.java	L34–39
getBaseRate() ()	LoanProduct abstracto	MortgageLoan → 7.5%	model/MortgageLoan.java	L21
getBaseRate() ()	LoanProduct abstracto	EducationalLoan → 5.0%	model/EducationalLoan.java	L12
getBaseRate() ()	LoanProduct abstracto	ConsumerLoan → 12.0%	model/ConsumerLoan.java	L12
getBaseRate() ()	LoanProduct abstracto	FreeInvestment Loan → 14.0%	model/FreeInvestmentLoan.java	L12
getMaxTerm ()	LoanProduct abstracto	MortgageLoan → 360 meses	model/MortgageLoan.java	L25

<code>getMaxTerm()</code>	<code>LoanProduct</code> abstracto	<code>EducationalLoan</code> → 120 meses	<code>model/EducationalLoan.java</code>	L13
<code>getMaxTerm()</code>	<code>LoanProduct</code> abstracto	<code>ConsumerLoan</code> → 60 meses	<code>model/ConsumerLoan.java</code>	L13
<code>getMaxTerm()</code>	<code>LoanProduct</code> abstracto	<code>FreeInvestmentLoan</code> → 48 meses	<code>model/FreeInvestmentLoan.java</code>	L13
<code>getDescription()</code>	<code>LoanProduct</code> abstracto	cada subtipo	cada archivo <code>model/</code>	L14
<code>getDescription()</code>	<code>LoanProduct</code> abstracto	Mortgage → "Long-term real-estate financing"	<code>model/MortgageLoan.java</code>	L30
Polimorfismo en GUI (runtime dispatch)	iteración <code>List<LoanProduct></code>	<code>getType()</code> , <code>getBaseRate()</code> , <code>getMaxTerm()</code>	<code>gui/DashboardPanel.java</code>	L216
Polimorfismo en GUI (runtime dispatch)	iteración <code>List<LoanProduct></code>	mismo	<code>gui/NewLoanPanel.java</code>	L174
Tratar <code>Client</code> como <code>Person</code> en loop	<code>printPersonSummaries()</code>	<code>getSummary()</code> y <code>getRole()</code> despachan a <code>Client</code>	<code>gui/ClientsPanel.java</code>	L228–235

3. ENCAPSULAMIENTO

Elemento	Modificador	Archivo	Línea
Atributos de <code>Person</code> (id, name, document...)	<code>protected</code>	<code>model/Person.java</code>	L18–22
<code>id</code> y <code>createdAt</code> son <code>final</code> (inmutables)	<code>protected final</code>	<code>model/Person.java</code>	L18, L22
<code>monthlyIncome</code> , <code>monthlyExpenses</code>	<code>private</code>	<code>model/Client.java</code>	L18–19

<code>loanIds</code> retorna copia inmutable	<code>Collections.unmodifiableList</code>	<code>model/Client.java</code>	L65
Todos los atributos de <code>Loan</code>	<code>private final</code>	<code>model/Loan.java</code>	L16–26
Atributos de <code>AmortizationRow</code>	<code>private final</code>	<code>model/AmortizationRow.java</code>	L11–15
Constructores de Singleton privados	<code>private</code>	<code>logic/LoanProductFactory.java</code>	L16
Constructores de Singleton privados	<code>private</code>	<code>registry/LoanRegistry.java</code>	L26

4. ABSTRACCIÓN

Elemento abstracto	Tipo	Archivo	Línea
<code>Person</code>	clase abstracta	<code>model/Person.java</code>	L16
<code>getRole()</code>	método abstracto en <code>Person</code>	<code>model/Person.java</code>	L32
<code>getSummary()</code>	método abstracto en <code>Person</code>	<code>model/Person.java</code>	L33
<code>LoanProduct</code>	clase abstracta	<code>model/LoanProduct.java</code>	L15
<code>getBaseRate()</code>	método abstracto en <code>LoanProduct</code>	<code>model/LoanProduct.java</code>	L57
<code>getMaxTerm()</code>	método abstracto en <code>LoanProduct</code>	<code>model/LoanProduct.java</code>	L58
<code>getDescription()</code>	método abstracto en <code>LoanProduct</code>	<code>model/LoanProduct.java</code>	L59
<code>IEvaluable</code>	interfaz	<code>interfaces/IEvaluable.java</code>	L8–11
<code>ILoanable</code>	interfaz	<code>interfaces/ILoanable.java</code>	L10–17

DESCRIPCIÓN DE CADA ARCHIVO

model/Person.java — Clase abstracta base

Propósito: Superclase abstracta común para `Client` y `LoanOfficer`.

Elemento	Tipo	Línea
Declaración <code>abstract class Person</code>	clase abstracta	L16
<code>id</code> (final, immutable)	atributo <code>protected final String</code>	L18
<code>name</code>	atributo <code>protected String</code>	L19
<code>document</code>	atributo <code>protected String</code>	L20
<code>email</code>	atributo <code>protected String</code>	L21
<code>createdAt</code> (fecha/hora de creación)	atributo <code>protected final LocalDateTime</code>	L22
Constructor protegido	<code>protected Person(id, name, document, email)</code>	L24–30
<code>getRole()</code>	método abstracto	L32
<code>getSummary()</code>	método abstracto	L33
<code>getDisplayName()</code>	retorna <code>name + "[" + document + "]"</code>	L35
<code>getHeader()</code>	retorna <code>"[ROLE] id name"</code>	L36
Getters y setters	<code>getId, getName, getDocument, getEmail, getCreatedAt</code>	L41–57

model/Client.java — Cliente financiero

Propósito: Extiende `Person`. Representa un cliente con ingresos y gastos.

Elemento	Tipo	Línea
<code>class Client extends Person</code>	herencia	L16
<code>monthlyIncome</code>	<code>private double</code>	L18
<code>monthlyExpenses</code>	<code>private double</code>	L19
<code>loanIds</code>	<code>private final List<String></code>	L20
<code>PAYMENT_CAPACITY_RATIO = 0.30</code>	constante (30%)	L22
Constructor	recibe id, name, document, email, income, expenses	L24–30
<code>getRole() → "CLIENT"</code>	override	L33
<code>getSummary()</code>	override, muestra income/score/loans	L38–44
<code>getNetIncome()</code>	<code>monthlyIncome - monthlyExpenses</code>	L47
<code>getMaxPaymentCapacity()</code>	<code>monthlyIncome * 0.30</code>	L48
<code>canAfford(pay)</code>	<code>pay <= getMaxPaymentCapacity()</code>	L49
<code>getCreditScore()</code>	calcula AAA/AA/A/BBB/C según ratio	L54–62
Umbrales del credit score	$\geq 0.50 \rightarrow \text{AAA}$, $\geq 0.35 \rightarrow \text{AA}$, $\geq 0.20 \rightarrow \text{A}$, $\geq 0.05 \rightarrow \text{BBB}$, otro $\rightarrow \text{C}$	L57–61
<code>addLoanId(loanId)</code>	agrega ID de préstamo	L64
<code>getLoanIds()</code>	retorna lista inmutable	L65

model/LoanOfficer.java — Oficial de crédito

Propósito: Extiende `Person`. Registra aprobaciones y rechazos.

Elemento	Tipo	Línea
<code>class LoanOfficer extends Person</code>	herencia	L13
<code>branch</code>	<code>private String</code> (sucursal)	L15
<code>approvedLoansCount</code>	<code>private int</code>	L16
<code>rejectedLoansCount</code>	<code>private int</code>	L17
Constructor	recibe id, name, document, email, branch	L19–25
<code>getRole() → "LOAN_OFFICER"</code>	override	L28
<code>getSummary()</code>	override	L34–39
<code>recordApproval()</code>	incrementa <code>approvedLoansCount++</code>	L41
<code>recordRejection()</code>	incrementa <code>rejectedLoansCount++</code>	L42

model/Loan.java — Préstamo

Propósito: Representa un préstamo con su tabla de amortización y estado.

Elemento	Tipo	Línea
<code>enum Status → PENDING, APPROVED, REJECTED</code>	enumeración interna	L14
<code>id, clientId, createdAt</code>	<code>private final</code>	L16–18
<code>product</code>	<code>private final LoanProduct</code>	L19
<code>principal</code>	capital solicitado	L20
<code>terms</code>	plazo en meses	L21
<code>annualRate</code>	tasa anual	L22

<code>monthlyPayment</code>	cuota mensual (calculada en constructor)	L23
<code>amortizationTable</code>	<code>List<AmortizationRow></code> generada en constructor	L24
<code>status</code>	estado mutable (PENDING/APPROVED/REJECTED)	L25
<code>evaluationResult</code>	resultado de evaluación crediticia	L26
Constructor	calcula <code>monthlyPayment</code> y genera tabla	L28–40
<code>monthlyPayment</code> calculado en línea	<code>product.calculateMonthlyPayment(..)</code>	L38
<code>amortizationTable</code> generada en línea	<code>product.generateAmortizationTable(...)</code>	L39
<code>approve(result)</code>	cambia status a APPROVED	L42–45
<code>reject(result)</code>	cambia status a REJECTED	L47–50
<code>getTotalCost()</code>	<code>monthlyPayment * terms</code>	L52
<code>getTotalInterest()</code>	<code>getTotalCost() - principal</code>	L53
<code>isApproved()</code>	<code>status == Status.APPROVED</code>	L54

`model/LoanProduct.java` — Producto de préstamo (abstracto)

Propósito: Clase abstracta base para los 4 tipos de préstamo. Implementa `ILoanable`.

Elemento	Tipo	Línea
<code>abstract class LoanProduct implements ILoanable</code>	abstracta + interfaz	L15
<code>type</code> e <code>icon</code>	<code>protected final String</code>	L17–18
<code>calculateMonthlyPayment(principal, annualRate, terms)</code>	fórmula de cuota fija (anualidad)	L26–32
Fórmula: <code>i = annualRate/100/12</code>	tasa mensual	L28

Fórmula: <code>factor = (1+i)^terms</code>		L30
Fórmula: <code>principal * (i * factor) / (factor - 1)</code>	cuota final	L31
<code>generateAmortizationTable(...)</code>	genera lista de <code>AmortizationRow</code>	L35–52
<code>getBaseRate()</code>	método abstracto (cada subclase lo define)	L57
<code>getMaxTerm()</code>	método abstracto	L58
<code>getDescription()</code>	método abstracto	L59

model/AmortizationRow.java — Fila de amortización

Propósito: Inmutable. Representa una fila de la tabla de amortización.

Elemento	Línea
<code>period</code> (número de cuota)	L11
<code>payment</code> (cuota total)	L12
<code>interest</code> (intereses)	L13
<code>capitalPayment</code> (abono a capital)	L14
<code>remainingBalance</code> (saldo restante)	L15
Constructor completo	L17–24
Todos los getters	L26–30

model/EvaluationResult.java — Resultado de evaluación

Propósito: Inmutable. Encapsula el resultado de evaluar un cliente para un préstamo.

Elemento	Tipo	Línea
<code>approved</code>	boolean	L11
<code>monthlyPayment</code>	cuota propuesta	L12
<code>maxCapacity</code>	capacidad máxima del cliente	L13
<code>paymentRatio</code>	ratio pago/ingreso	L14
<code>reason</code>	texto explicativo del resultado	L15
<code>evaluatorName</code>	nombre del evaluador	L16
Constructor	L18–27	
<code>getPaymentRatioFormatted()</code>	formatea como "xx.x%"	L36–38

model/ConsumerLoan.java — Préstamo de consumo

Elemento	Valor	Línea
<code>super("CONSUMER", </code>)	tipo e ícono	L10
<code>getBaseRate()</code>	12.0%	L12
<code>getMaxTerm()</code>	60 meses	L13
<code>getDescription()</code>	"General consumer goods financing"	L14

model/EducationalLoan.java — Préstamo educativo

Elemento	Valor	Línea
<code>super("EDUCATIONAL", "🎓")</code>	tipo e ícono	L10
<code>getBaseRate()</code>	5.0%	L12
<code>getMaxTerm()</code>	120 meses	L13
<code>getDescription()</code>	"Student and academic financing"	L14

model/MortgageLoan.java — Préstamo hipotecario

Elemento	Valor	Línea
<code>super("MORTGAGE", "🏠")</code>	tipo e ícono	L16
<code>getBaseRate()</code>	7.5%	L21
<code>getMaxTerm()</code>	360 meses (30 años)	L25
<code>getDescription()</code>	"Long-term real-estate financing"	L30

model/FreeInvestmentLoan.java — Préstamo libre inversión

Elemento	Valor	Línea
<code>super("FREE_INVESTMENT", "💼")</code>	tipo e ícono	L10
<code>getBaseRate()</code>	14.0%	L12
<code>getMaxTerm()</code>	48 meses	L13
<code>getDescription()</code>	"Flexible-purpose personal lending"	L14

interfaces/IEvaluable.java — Interfaz de evaluación

Elemento	Línea
evaluate(Client client, double monthlyPayment)	L9
getCapacityThreshold()	L10
getEvaluatorName()	L11

interfaces/ILoanable.java — Interfaz de producto de préstamo

Elemento	Línea
calculateMonthlyPayment(principal, annualRate, terms)	L11
generateAmortizationTable(principal, annualRate, terms)	L12
getMaxTerm()	L13
getBaseRate()	L14
getDescription()	L15
getType()	L16
getIcon()	L17

logic/CreditEvaluator.java — Evaluador de crédito

Propósito: Implementa `IEvaluable`. Decide si un préstamo se aprueba o rechaza.

Elemento	Tipo	Línea
implements IEvaluable	interfaz	L13
DEFAULT_THRESHOLD = 0.30	umbral por defecto (30%)	L15
capacityThreshold	campo privado configurable	L16
Constructor por defecto	usa 0.30	L18–20

Constructor con parámetro	permite umbral personalizado	L22–24
<code>evaluate(client, monthlyPayment)</code>	lógica principal de aprobación	L27–43
Lógica: <code>approved = monthlyPayment <= maxCapacity</code>		L32
Mensaje APPROVED	formato del mensaje	L35–36
Mensaje REJECTED	incluye exceso en \$	L37–39
<code>getEvaluatorName()</code> → "StandardCreditEvaluator"		L46

`logic/LoanProductFactory.java` — Fábrica de productos (Singleton)

Propósito: Patrón Singleton + Factory. Crea y entrega los 4 productos de préstamo.

Elemento	Tipo	Línea
<code>private static LoanProductFactory instance</code>	Singleton	L13
<code>catalog</code>	<code>Map<String, LoanProduct></code> (LinkedHashMap)	L14
Constructor privado	registra los 4 productos	L16–22
Se registran: Mortgage, Educational, Consumer, FreeInvestment		L18–21
<code>getInstance()</code>	retorna instancia única	L24–27
<code>getProduct(type)</code>	retorna producto por tipo string	L33
<code>getAllProducts()</code>	lista inmutable de todos los productos	L36–38
<code>getProductTypes()</code>	set de strings con los tipos	L40–42

registry/LoanRegistry.java — Registro central (Singleton)

Propósito: Patrón Singleton. Repositorio central de clientes y préstamos. Punto de entrada de toda la lógica.

Elemento	Tipo	Línea
<code>private static LoanRegistry instance</code>	Singleton	L16
<code>clients</code>	<code>Map<String, Client></code>	L18
<code>loans</code>	<code>Map<String, Loan></code>	L19
<code>productFactory</code>	<code>LoanProductFactory</code>	L20
<code>evaluator</code>	<code>IEvaluable</code> (inyectable)	L21
<code>clientSeq, loanSeq</code>	contadores para IDs	L23–24
Constructor privado	inicializa y llama <code>seedData()</code>	L26–32
<code>getInstance()</code>	retorna instancia única	L34–37
<code>seedData()</code>	precarga 4 clientes y 4 préstamos	L39–53
Clientes precargados	Ana, Carlos, Laura, Diego	L40–47
Préstamos precargados	Mortgage(Ana), Educational(Carlos), Consumer(Laura), FreeInvestment(Diego)	L49–52
<code>registerClient(name, document, email, income, expenses)</code>	crea Client con ID auto "C0001..."	L55–62
ID generado como	<code>String.format("C%04d", clientSeq++)</code>	L57
<code>requestLoan(clientId, productType, principal, terms, customRate)</code>	crea Loan, evalúa, aprueba/rechaza	L64–81
Lógica de tasa	usa <code>customRate</code> si no es null, sino <code>product.getBaseRate()</code>	L68

ID de préstamo generado como	<code>String.format("L%04d", loanSeq++)</code>	L69
Evaluación automática	<code>evaluator.evaluate(client, loan.getMonthlyPayment())</code>	L73
<code>setEvaluator(evaluator)</code>	permite inyectar evaluador distinto	L94–96

gui/Main.java — Punto de entrada

Elemento	Línea
<code>public static void main(String[] args)</code>	L10
Crea <code>MainFrame</code> y lo hace visible	L12–13

gui/MainFrame.java — Ventana principal

Propósito: Ventana JFrame con sidebar de navegación y CardLayout central.

Elemento	Línea
<code>setTitle("FinCredit – Credit & Loan Management System")</code>	L33
<code>setSize(1200, 700)</code>	L35
<code>setMinimumSize(900, 600)</code>	L37
Colores constantes (BG_NAVY, RED_ACCENT, etc.)	L20–28
<code>buildSidebar()</code>	construye panel izquierdo de navegación
Botones de navegación	Dashboard, Clients, Loans, New Client, Request Loan

<code>buildCenter()</code>	construye panel principal con CardLayout
<code>cardLayout = new CardLayout()</code>	manejo de vistas
Cards registradas	"dashboard", "clients", "newClient", "loans", "newLoan"
ActionListeners de navegación	botones muestran card correspondiente

`gui/DashboardPanel.java` — Panel Dashboard

Propósito: Muestra estadísticas generales y catálogo de productos.

Elemento	Línea
<code>registry = LoanRegistry.getInstance()</code>	acceso al registro
<code>buildStatCards()</code>	4 tarjetas de resumen
Tarjeta "Total Clients"	<code>registry.getAllClients().size()</code>
Tarjeta "Loan Requests"	<code>loans.size()</code>
Tarjeta "Approval Rate"	<code>approved * 100.0 / totalLoans</code>
Tarjeta "Active Portfolio"	suma de <code>getPrincipal()</code> de aprobados
<code>buildProductCatalog()</code>	lista de productos con polimorfismo
Iteración polimórfica de productos	<code>LoanProductFactory.getInstance().getAllProducts()</code>
<code>buildLoanTable()</code>	tabla de préstamos recientes
Renderer color Status (APPROVED/REJECTED)	

`gui/ClientsPanel.java` — Panel Lista de Clientes

Elemento	Línea
Tabla con columnas: ID, Full Name, Document, Monthly Income, Payment Capacity, Credit Score, Loans	L129–132

<code>client.getCreditScore()</code> → colorear AAA/AA/A/BBB/C	L170–177
Renderer colores: AAA=verde, AA=verde oscuro, A=azul, BBB=naranja, C=rojo	L171–176
Botón "+ New Client" navega a "newClient"	L79
<code>printPersonSummaries()</code> — demo polimorfismo	L228–235
En ese método: <code>List<Client></code> tratada como <code>List<Person></code>	L230

gui/LoansPanel.java — Panel Lista de Préstamos

Elemento	Línea
Tabla con columnas: Loan ID, Client, Product Type, Principal, Rate %, Term, Monthly Pay, Total Cost, Status	L134–138
Conteo de aprobados y rechazados en header	L60–65
Renderer color Status	L175–191
Renderer color por tipo de producto	L196–213
Botón "+ Request Loan" navega a "newLoan"	L85

gui/NewClientPanel.java — Formulario Nuevo Cliente

Elemento	Línea
Campos del formulario: txtName, txtDocument, txtEmail, txtIncome, txtExpenses	L28–32
<code>makeFieldGroup(label, placeholder)</code> — crea campo con placeholder	L170–226
Asignación de campos según label	L216–220
<code>registerClient()</code> — lógica principal de registro	L306–377

Validación campos vacíos	L316–320
Validación de email (contiene @ y .)	L323–326
<code>NumberFormatException</code> al parsear income/expenses	L332–339
Validación: income > 0	L342–345
Validación: expenses >= 0	L347–350
Validación: expenses < income	L352–355
Llamada a <code>registry.registerClient(...)</code>	L359–361
Mensaje de éxito con ID nuevo	L364–365
Timer 1500ms → navega a "clients"	L369–371
Regla de negocio explicada en InfoBox	L244–245 (30% de ingreso)

`gui/NewLoanPanel.java` — Formulario Nuevo Préstamo

Elemento	Línea
Campos: cmbClient, cmbProduct, txtPrincipal, txtTerms, txtCustomRate	L33–37
Preview en tiempo real: lblPayment, lblTotalCost, lblTotalInterest, lblCapacity, lblEvaluation	L40–45
Cargar clientes en combo	L155–158 (iterar <code>registry.getAllClients()</code>)
Cargar productos en combo (polimorfismo)	L174–178 (iterar <code>productFactory.getAllProducts()</code>)
<code>updatePreview()</code> — calcula preview en tiempo real	L291–354
Llamada polimórfica: <code>product.calculateMonthlyPayment(...)</code>	L324
Pre-aprobación: <code>client.canAfford(payment)</code>	L333
Mensaje "✓ Pre-approved" / "✗ Pre-rejected"	L344 / L347

<code>submitLoan()</code> — lógica de envío	L412–505
Validación <code>term > product.getMaxTerm()</code> (polimórfico)	L463–467
Llamada a <code>registry.requestLoan(...)</code>	L478–484
Resultado APPROVED / REJECTED	L487–493
<code>NumberFormatException</code> capturado en submit	L439–442
Timer 1500ms → navega a "loans"	L497–499

PREGUNTAS FRECUENTES Y SUS RESPUESTAS EN CÓDIGO

Pregunta probable	Respuesta	Archivo	Línea
¿Dónde está la herencia?	<code>Client extends Person</code>	<code>model/Client.java</code>	L16
¿Dónde está el polimorfismo?	<code>getBaseRate()</code> en subtipos de <code>LoanProduct</code>	<code>model/ConsumerLoan.java</code> etc.	L12
¿Qué es <code>Person</code> ?	Clase abstracta con <code>getRole()</code> y <code>getSummary()</code> abstractos	<code>model/Person.java</code>	L16
¿Cómo se calcula la cuota?	Fórmula de anualidad en <code>calculateMonthlyPayment</code>	<code>model/LoanProduct.java</code>	L26–32
¿Cómo se aprueba un préstamo?	Cuota $\leq$ 30% del ingreso mensual	<code>logic/CreditEvaluator.java</code>	L32
¿Qué es el credit score?	Ratio net income / income → AAA a C	<code>model/Client.java</code>	L54–62
¿Qué es el Singleton?	<code>LoanRegistry</code> y <code>LoanProductFactory</code>	<code>registry/LoanRegistry.java</code>	L16, L34
¿Cómo funciona la fábrica?	<code>LoanProductFactory.getProduct(type)</code>	<code>logic/LoanProductFactory.java</code>	L33
¿Qué datos vienen precargados?	4 clientes y 4 préstamos en <code>seedData()</code>	<code>registry/LoanRegistry.java</code>	L39–53

¿Cómo se genera la tabla de amortización?	<code>generateAmortizationTable()</code> en <code>LoanProduct</code>	<code>model/LoanProduct.java</code>	L35–52
¿Qué interfaz implementa <code>CreditEvaluator</code> ?	<code>IEvaluable</code>	<code>logic/CreditEvaluator.java</code>	L13
¿Qué interfaz implementa <code>LoanProduct</code> ?	<code>ILoanable</code>	<code>model/LoanProduct.java</code>	L15
¿Dónde se maneja el error de número inválido?	<code>NumberFormatException</code>	<code>gui/NewClientPanel.java</code>	L335
¿Cuál es el punto de entrada?	<code>Main.main()</code> crea <code>MainFrame</code>	<code>gui/Main.java</code>	L10–13
¿Cómo navega entre pantallas?	<code>CardLayout.show(mainPanel, "nombre")</code>	<code>gui/MainFrame.java</code>	L227–231



TIPOS DE PRÉSTAMO — RESUMEN RÁPIDO

Tipo	Tasa base	Plazo máx.	Clase	Archivo
MORTGAGE (🏠)	7.5%	360 meses	<code>MortgageLoan</code>	<code>model/MortgageLoan.java</code>
EDUCATIONAL (🎓)	5.0%	120 meses	<code>EducationalLoan</code>	<code>model/EducationalLoan.java</code>
CONSUMER (🛒)	12.0%	60 meses	<code>ConsumerLoan</code>	<code>model/ConsumerLoan.java</code>
FREE_INVESTMENT (💼)	14.0%	48 meses	<code>FreeInvestmentLoan</code>	<code>model/FreeInvestmentLoan.java</code>